Module 1 – Scalability

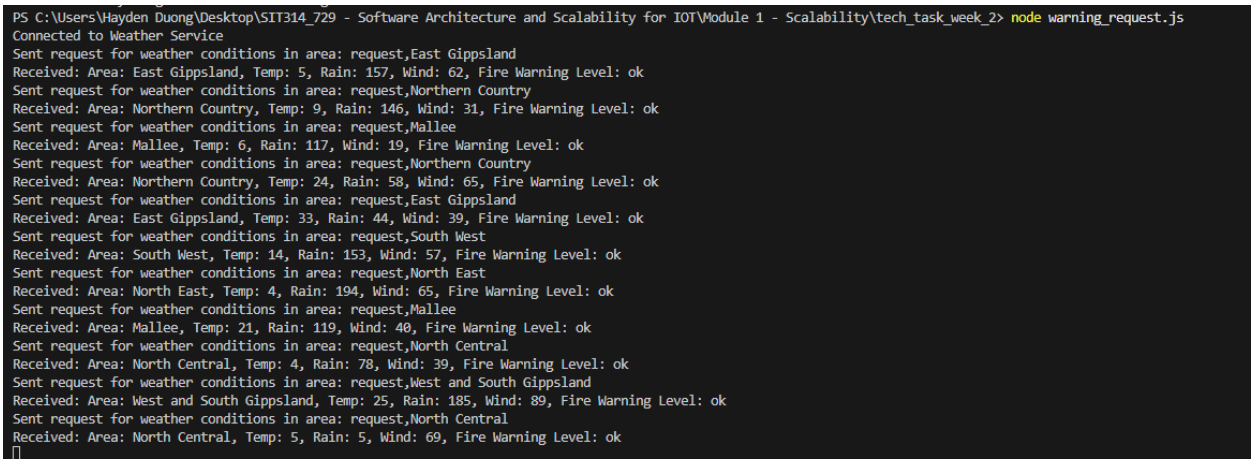
Code for modified warning_request.js:

```
// warning_request.js

// This code implements a TCP socket client that sends a request for weather conditions

// and listens for responses from the server.

const net = require("net");


const host = "127.0.0.1";

const port = 6000;


// Specify the area for which to request weather conditions

// List of areas can be defined based on the fire warning levels:

// Central, East Gippsland, Mallee, North Central, North East, Northern Country, South West, West and South Gippsland, Wimmera

const areas = [

  "Central",

  "East Gippsland",

  "Mallee",

  "North Central",

  "North East",

  "Northern Country",

  "South West",

  "West and South Gippsland",

  "Wimmera"

];
```

```
// const areaToRequest = "Central"; // Specify the area for which to request weather conditions


// Create a TCP client that connects to the server
const client = net.createConnection(port, host, () => {
  console.log("Connected to Weather Service");


  // Set the encoding for the client
  setInterval(() => {
    const areaToRequest = areas[Math.floor(Math.random() * areas.length)]; // Randomly select an area from the list
    client.write(`request,${areaToRequest}`);
    console.log(`Sent request for weather conditions in area: request,${areaToRequest}`);
  }, 2000); // Interval in milliseconds (2000ms = 2 seconds)
});


// Handle incoming data from the server
client.on("data", (data) => {
  console.log(`Received: ${data.toString()}`);
//   process.exit(0);
});


// Handle socket error event
client.on("error", (error) => {
  console.log(`Error: ${error.message}`);
});
```

```
// Handle socket end event

client.on("close", () => {

   console.log("Connection closed");

});
```

Picture 1 – Results obtained from weather_service.js



Picture 2 – Results obtained from warning_request.js

Scalability Issues with the Current Architecture

The current implementation of the IoT weather warning service is built upon a simple client-server model, where all sensor nodes and user request nodes connect directly to a single weather_service.js server.

To simulate a more realistic environment for this task, the three weather-sensing nodes (wind, rain, and temperature) have been configured to generate a random value every 2 seconds, which is then sent to the server.

The fire_node.js, however, is set to an interval of 300,000ms (5 minutes) to avoid being blocked by the CFA website, from which it scrapes real fire warning data.

On the client side, the warning_request.js code has been modified to randomly select a location from a list of Victorian fire districts and send a request to the server every 2 seconds, effectively simulating hundreds of users concurrently accessing the service.

While this setup demonstrates the system's basic functionality, a key limitation is the reliance on randomly generated data for temperature, rainfall, and wind, as a real-time data source could not be identified for these parameters.

This contrasts with the fire data, which is obtained from a live source. This simple architecture, while functional for a limited scope, presents significant scalability challenges when the system needs to handle a larger number of devices and users.

On the other hand, these following can be expected from my current implementation:

- **Single Point of Failure (SPOF):**
    - The most critical flaw of the current design is that the central server acts as a single point of failure.
    - If the weather_service.js application or the physical machine it runs on fails, the entire system becomes inoperable.
    - Both data collection from sensors and the ability to respond to user requests are completely halted.
    - In a critical service like a weather warning system, this lack of resilience is unacceptable, as it could lead to missed warnings and potential safety risks.

- **Resource Bottleneck:**
    - As the number of sensor nodes and user requests increases, the single server becomes a resource bottleneck.
    - It must manage a growing number of simultaneous TCP connections and process all incoming data and outgoing responses.

    - This leads to a strain on the server's CPU, memory, and network resources.
        - As a result, the system's performance will degrade, latency will increase, and at a certain point, the server will become overwhelmed and unable to respond reliably.
    - This limits the system's ability to scale horizontally, as all processing is concentrated on a single machine.

- **Lack of Data Persistence:**
- The current system stores all sensor data in volatile in-memory variables.
- If the server were to crash or be restarted, all collected weather data would be lost. This prevents any form of historical data analysis, which is crucial for identifying long-term trends, improving warning accuracy, or providing post-event analysis.

Proposed Architectural Improvements for Scalability

To address the issues and create a more robust and scalable system, a shift from the current centralized model to a distributed architecture is required. The following improvements are proposed to enhance the system's resilience, performance, and scalability.

- **Decoupling with a Message Queue:**
    - To eliminate the single point of failure and reduce the load on the central server, a **Message Queue** (e.g., RabbitMQ, Apache Kafka) should be introduced. In this new model, sensor nodes would become **producers**, publishing their data to the message queue.
    - The weather_service.js application would become a **consumer**, subscribing to the data streams from the queue. This approach decouples the producers from the consumers.
    - If the weather_service.js is temporarily unavailable, sensor data can still be queued and processed once the service is restored, ensuring no data is lost. This also offloads the burden of managing numerous connections from the server to the message broker.

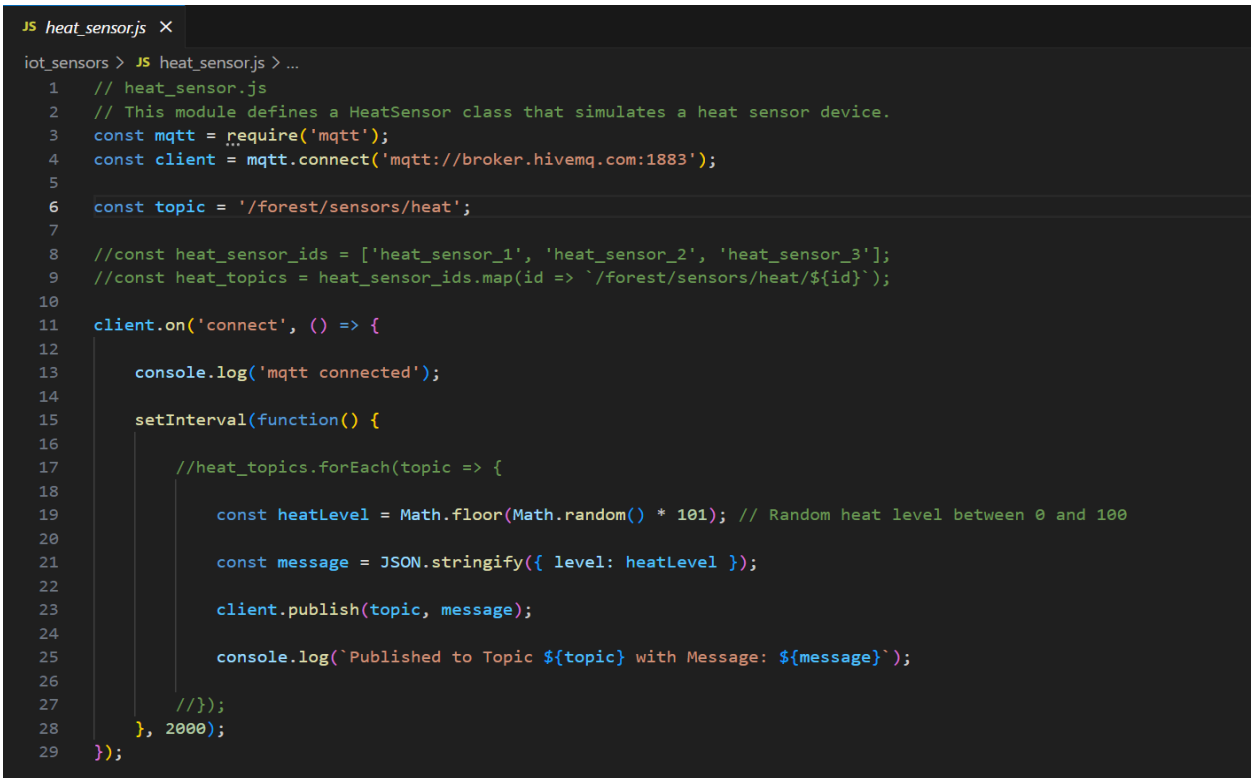
- **Integrating a Database for Data Persistence:**
    - To ensure data durability and enable historical analysis, a dedicated database should be integrated into the architecture.
    - The weather_service.js application would be modified to write all incoming sensor data to the database instead of storing it in memory.

    - This ensures that even in the event of a system failure, all collected data is preserved.
    - A time-series database (e.g., InfluxDB) would be particularly well-suited for this type of time-stamped sensor data.

- **Implementing Horizontal Scaling with a Load Balancer:**
    - To overcome the resource bottleneck and further increase fault tolerance, the weather_service.js should be horizontally scaled.
        - This involves running multiple instances of the server behind a **Load Balancer**.
        - The load balancer would intelligently distribute incoming requests from user nodes across the available server instances.
        - This not only increases the system's capacity to handle a high volume of concurrent requests but also provides redundancy.
        - If one of the server instances fails, the load balancer will simply route traffic to the remaining healthy servers, providing a seamless and uninterrupted service to users.

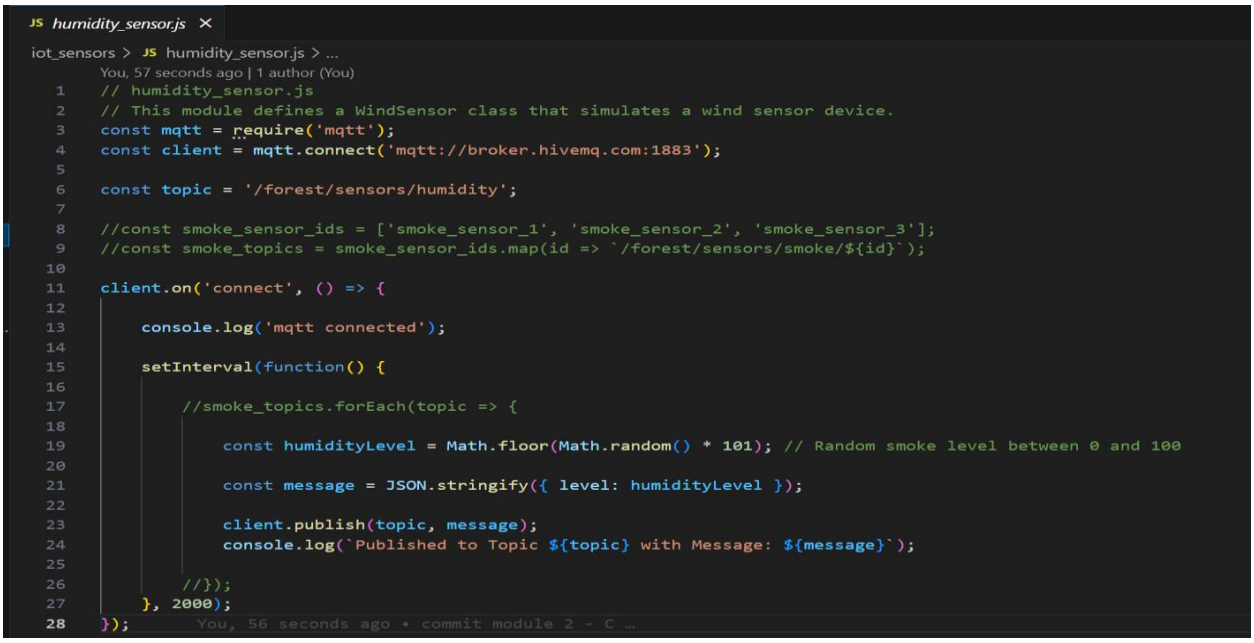
Module 2 – Scalable IoT Applications

Screenshots



```
JS heat_sensor.js ×
iot_sensors > JS heat_sensor.js > ...
1   // heat_sensor.js
2   // This module defines a HeatSensor class that simulates a heat sensor device.
3   const mqtt = require('mqtt');
4   const client = mqtt.connect('mqtt://broker.hivemq.com:1883');
5
6   const topic = '/forest/sensors/heat';
7
8   //const heat_sensor_ids = ['heat_sensor_1', 'heat_sensor_2', 'heat_sensor_3'];
9   //const heat_topics = heat_sensor_ids.map(id => `/forest/sensors/heat/${id}`);
10
11  client.on('connect', () => {
12
13      console.log('mqtt connected');
14
15      setInterval(function() {
16
17          //heat_topics.forEach(topic => {
18
19              const heatLevel = Math.floor(Math.random() * 101); // Random heat level between 0 and 100
20
21              const message = JSON.stringify({ level: heatLevel });
22
23              client.publish(topic, message);
24
25              console.log(`Published to Topic ${topic} with Message: ${message}`);
26
27          //});
28      }, 2000);
29  });
```

Picture 3 – Code of heat_sensor.js



```
JS humidity_sensor.js ×
iot_sensors > JS humidity_sensor.js > ...
    You, 57 seconds ago | 1 author (You)
1   // humidity_sensor.js
2   // This module defines a WindSensor class that simulates a wind sensor device.
3   const mqtt = require('mqtt');
4   const client = mqtt.connect('mqtt://broker.hivemq.com:1883');
5
6   const topic = '/forest/sensors/humidity';
7
8   //const smoke_sensor_ids = ['smoke_sensor_1', 'smoke_sensor_2', 'smoke_sensor_3'];
9   //const smoke_topics = smoke_sensor_ids.map(id => `/forest/sensors/smoke/${id}`);
10
11  client.on('connect', () => {
12
13      console.log('mqtt connected');
14
15      setInterval(function() {
16
17          //smoke_topics.forEach(topic => {
18
19              const humidityLevel = Math.floor(Math.random() * 101); // Random smoke level between 0 and 100
20
21              const message = JSON.stringify({ level: humidityLevel });
22
23              client.publish(topic, message);
24              console.log(`Published to Topic ${topic} with Message: ${message}`);
25
26          //});
27      }, 2000);
28  });      You, 56 seconds ago • commit module 2 - C …
```

Picture 4 – Code of humidity_sensor.js

```
JS smoke_sensor.js ×
iot_sensors > JS smoke_sensor.js > ...
You, 2 minutes ago | 1 author (You)
1   // smoke_sensor.js
2   // This module defines a SmokeSensor class that simulates a smoke sensor device.
3   const mqtt = require('mqtt');
4   const client = mqtt.connect('mqtt://broker.hivemq.com:1883');
5
6   const topic = '/forest/sensors/smoke';
7
8   //const smoke_sensor_ids = ['smoke_sensor_1', 'smoke_sensor_2', 'smoke_sensor_3'];
9   //const smoke_topics = smoke_sensor_ids.map(id => `/forest/sensors/smoke/${id}`);
10
11  client.on('connect', () => {
12
13      console.log('mqtt connected');
14
15      setInterval(function() {
16
17          //smoke_topics.forEach(topic => {
18
19              const smokeLevel = Math.floor(Math.random() * 101); // Random smoke level between 0 and 100
20
21              const message = JSON.stringify({ level: smokeLevel });
22
23              client.publish(topic, message);
24              console.log(`Published to Topic ${topic} with Message: ${message}`);
25
26          //});
27      }, 2000);
28  });
```

Picture 5 – Code of smoke_sensor.js

```
JS wind_sensor.js ×
iot_sensors > JS wind_sensor.js > ...
You, 2 minutes ago | 1 author (You)
1   // wind_sensor.js
2   // This module defines a WindSensor class that simulates a wind sensor device.
3   const mqtt = require('mqtt');
4   const client = mqtt.connect('mqtt://broker.hivemq.com:1883');
5
6   const topic = '/forest/sensors/wind';
7
8   //const smoke_sensor_ids = ['smoke_sensor_1', 'smoke_sensor_2', 'smoke_sensor_3'];
9   //const smoke_topics = smoke_sensor_ids.map(id => `/forest/sensors/smoke/${id}`);
10
11  client.on('connect', () => {
12
13      console.log('mqtt connected');
14
15      setInterval(function() {
16
17          //smoke_topics.forEach(topic => {
18
19              const windLevel = Math.floor(Math.random() * 101); // Random smoke level between 0 and 100
20
21              const message = JSON.stringify({ level: windLevel });
22
23              client.publish(topic, message);
24              console.log(`Published to Topic ${topic} with Message: ${message}`);
25
26          //});
27      }, 2000);
28  });      You, 2 minutes ago • commit module 2 - C
```

Picture 6 – Code of wind_sensor.js

fire_departments.js

clients > fire_departments.js > client.on('connect') callback

You, 3 minutes ago | 1 author (You)

```
// fire_departments.js
// This module defines a FireDepartments class that simulates a fire department client.
const mqtt = require('mqtt');
const client = mqtt.connect('mqtt://broker.hivemq.com:1883');

const topic = '/alert/fire_departments';

client.on('connect', () => {       You, 3 minutes ago • commit module 2 - C …
    client.subscribe(topic);
    console.log('Fire Department Client connected and subscribed to topic:', topic);
});

client.on('message', (topic, message) => {
    console.log(`Fire Department Client received message on topic ${topic}: ${message.toString()}`);
});
```

Picture 7 – Code of fire_departments.js

local_homeowners.js

clients > local_homeowners.js > ...

You, 3 minutes ago | 1 author (You)

```
// local_homeowners.js
// This module defines a LocalHomeowners class that simulates a local homeowner client.
const mqtt = require('mqtt');
const client = mqtt.connect('mqtt://broker.hivemq.com:1883');

const topic = '/alert/local_homeowners';

client.on('connect', () => {
    client.subscribe(topic);
    console.log('Local Homeowners Client connected and subscribed to topic:', topic);
});

client.on('message', (topic, message) => {
    console.log(`Local Homeowners Client received message on topic ${topic}: ${message.toString()}`);
});
```

Picture 8 – Code of the local_homeowners.js

news_organizations.js

clients > news_organizations.js > ...

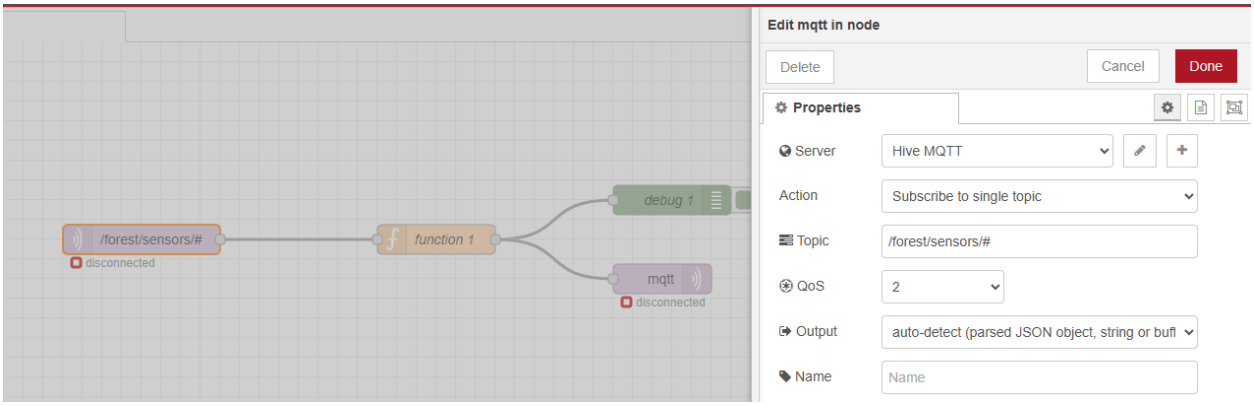
You, 4 minutes ago | 1 author (You)

```
// news_organizations.js
// This module defines a NewsOrganizations class that simulates a news organization client.
const mqtt = require('mqtt');
const client = mqtt.connect('mqtt://broker.hivemq.com:1883');

const topic = '/alert/news_organizations';

client.on('connect', () => {
    client.subscribe(topic);
    console.log('News Organization Client connected and subscribed to topic:', topic);
});

client.on('message', (topic, message) => {
    console.log(`News Organization Client received message on topic ${topic}: ${message.toString()}`);
});       You, 4 minutes ago • commit module 2 - C …
```

Picture 9 – Code of the news_organizations.js

```
JS social_media.js ×
clients > JS social_media.js > ...
      You, 4 minutes ago | 1 author (You)
1   // social_media.js
2   // This module defines a SocialMedia class that simulates a social media client.
3   const mqtt = require('mqtt');
4   const client = mqtt.connect('mqtt://broker.hivemq.com:1883');
5
6   const topic = '/alert/social_media';
7
8   client.on('connect', () => {
9       client.subscribe(topic);
10      console.log('Social Media Client connected and subscribed to topic:', topic);
11  });
12
13  client.on('message', (topic, message) => {
14      console.log(`Social Media Client received message on topic ${topic}: ${message.toString()}`);
15  });      You, 4 minutes ago • commit module 2 - C …
```

Picture 10 – Code from social_media.js



Picture 11 – MQTT Input Node on Node-RED.

Code for "function" node on Node-RED

```
// Use flow concept to store variables send from IoT Sensors
// Create an empty object if 'alertStates' has not existed before.
var flowContext = flow.get('alertStates') || {};

// Split the incoming msg into parts for further logic implementations
var parts = msg.topic.split('/');
var sensorType = parts[3];
var alertFactor = 0;
var sensorLevel = msg.payload.level;

// Store the level of the current sensor in flow context
```

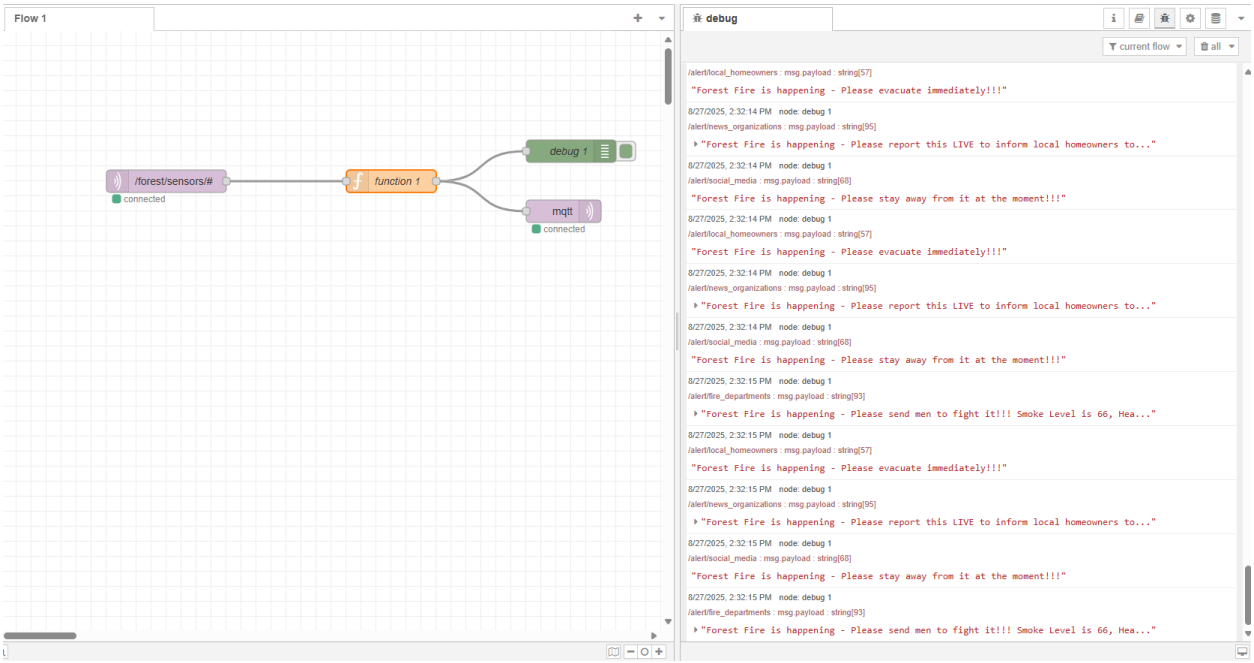
```
// This allows you to access the latest value from any sensor
flowContext[sensorType] = { level: sensorLevel, alert: 0 };

if (sensorType === "smoke") {
    if (sensorLevel >= 50) {
        flowContext[sensorType].alert = 1;
    }
} else if (sensorType === "heat") {
    if (sensorLevel >= 60) {
        flowContext[sensorType].alert = 1;
    }
} else if (sensorType === "wind") {
    if (sensorLevel >= 70) {
        flowContext[sensorType].alert = 1;
    }
} else if (sensorType === "humidity") {
    if (sensorLevel <= 20) {
        flowContext[sensorType].alert = 1;
    }
}

// Save the updated flow context
flow.set('alertStates', flowContext);

// Calculate the total number of alerts
var totalAlerts = 0;
for (var key in flowContext) {
    totalAlerts += flowContext[key].alert;
}

// Define an array to store warning messages
var alertsToSend = [];

// Main Alert Condition (>= 2 factors)
if (totalAlerts >= 2) {
    alertsToSend.push({
        topic: "/alert/local_homeowners",
        payload: `Forest Fire is happening - Please evacuate immediately!!!`
    });

    alertsToSend.push({
        topic: "/alert/news_organizations",
        payload: `Forest Fire is happening - Please report this LIVE to inform
local homeowners to be cautious!!!`
    });
```

```
    alertsToSend.push({
        topic: "/alert/social_media",
        payload: `Forest Fire is happening - Please stay away from it at the
moment!!!`
    });
}

// Specific Alert for Fire Departments
var currentSmokeLevel = flowContext['smoke'] ? flowContext['smoke'].level : 0;
var currentHeatLevel = flowContext['heat'] ? flowContext['heat'].level : 0;

if (currentSmokeLevel >= 50 && currentHeatLevel >= 80) {
    alertsToSend.push({
        topic: "/alert/fire_departments",
        payload: `Forest Fire is happening - Please send men to fight it!!! Smoke
Level is ${currentSmokeLevel}, Heat Level is ${currentHeatLevel}`
    });
}

// Send alerts if any are present
if (alertsToSend.length > 0) {
    return [alertsToSend];
}

return null;
```

Results:

```
[4] Published to Topic /forest/sensors/heat with Message: {"level":88}
[7] Published to Topic /forest/sensors/wind with Message: {"level":93}
[5] Published to Topic /forest/sensors/humidity with Message: {"level":42}
[6] Published to Topic /forest/sensors/smoke with Message: {"level":34}
[1] Local Homeowners Client received message on topic /alert/local_homeowners: Forest Fire is happening - Please evacuate immediately!!!
[3] Social Media Client received message on topic /alert/social_media: Forest Fire is happening - Please stay away from it at the moment!!!
[2] News Organization Client received message on topic /alert/news_organizations: Forest Fire is happening - Please report this LIVE to inform local hom
eowners to be cautious!!!
[1] Local Homeowners Client received message on topic /alert/local_homeowners: Forest Fire is happening - Please evacuate immediately!!!
[1] Local Homeowners Client received message on topic /alert/local_homeowners: Forest Fire is happening - Please evacuate immediately!!!
[3] Social Media Client received message on topic /alert/social_media: Forest Fire is happening - Please stay away from it at the moment!!!
[2] News Organization Client received message on topic /alert/news_organizations: Forest Fire is happening - Please report this LIVE to inform local hom
eowners to be cautious!!!
[2] News Organization Client received message on topic /alert/news_organizations: Forest Fire is happening - Please report this LIVE to inform local hom
eowners to be cautious!!!
[3] Social Media Client received message on topic /alert/social_media: Forest Fire is happening - Please stay away from it at the moment!!!
[4] Published to Topic /forest/sensors/heat with Message: {"level":46}
[7] Published to Topic /forest/sensors/wind with Message: {"level":3}
[5] Published to Topic /forest/sensors/humidity with Message: {"level":0}
[6] Published to Topic /forest/sensors/smoke with Message: {"level":47}
[2] News Organization Client received message on topic /alert/news_organizations: Forest Fire is happening - Please report this LIVE to inform local hom
eowners to be cautious!!!
[3] Social Media Client received message on topic /alert/social_media: Forest Fire is happening - Please stay away from it at the moment!!!
[1] Local Homeowners Client received message on topic /alert/local_homeowners: Forest Fire is happening - Please evacuate immediately!!!
[7] Published to Topic /forest/sensors/wind with Message: {"level":81}
[4] Published to Topic /forest/sensors/heat with Message: {"level":77}
[6] Published to Topic /forest/sensors/smoke with Message: {"level":5}
[5] Published to Topic /forest/sensors/humidity with Message: {"level":10}
[1] Local Homeowners Client received message on topic /alert/local_homeowners: Forest Fire is happening - Please evacuate immediately!!!
[1] Local Homeowners Client received message on topic /alert/local_homeowners: Forest Fire is happening - Please evacuate immediately!!!
[3] Social Media Client received message on topic /alert/social_media: Forest Fire is happening - Please stay away from it at the moment!!!
[3] Social Media Client received message on topic /alert/social_media: Forest Fire is happening - Please stay away from it at the moment!!!
[2] News Organization Client received message on topic /alert/news_organizations: Forest Fire is happening - Please report this LIVE to inform local hom
eowners to be cautious!!!
[2] News Organization Client received message on topic /alert/news_organizations: Forest Fire is happening - Please report this LIVE to inform local hom
eowners to be cautious!!!
```

Picture 12 – Results obtained from the client-side (all clients: fire_departments.js, local_homeowners.js, news_organizations.js, social_media.js).



Picture 13 – Results yielded from “debug” node in Red-NODE.

Explanations:

This project, a prototype of a forest fire monitoring and alarm system, was built using Node.js, MQTT, and Node-RED.

The core objective was to demonstrate a scalable and intelligent IoT solution capable of processing data from multiple sensors and delivering tailored alerts to different stakeholder groups, namely local homeowners, news organizations, social media, and fire departments.

1. Solution Overview

The solution is structured as a robust, event-driven system:

- **Sensors**: Simulated using Node.js scripts, these act as data publishers, sending mock sensor readings (e.g., smoke, heat, wind, humidity levels) to an MQTT broker.
- **Node-RED**: This acts as the central processing unit, subscribing to the raw sensor data, applying complex business logic, and making decisions on which alerts to send.
- **Clients**: Simulated as separate Node.js scripts, these act as subscribers, listening to specific alerts on dedicated MQTT topics.

2. Key Components and Their Functionality

a) MQTT Client Scripts (Sensors & Clients)

We implemented two main types of Node.js scripts. Both connect to the public MQTT broker broker.hivemq.com and operate asynchronously.

- **Sensor Scripts**: Scripts like smoke_sensor.js and heat_sensor.js continuously publish messages to specific, hierarchical topics (e.g., /forest/sensors/smoke and /forest/sensors/heat). The hierarchical topic structure allows the system to easily differentiate between sensor types.
- **Client Scripts**: Each client file (e.g., local_homeowners.js, fire_departments.js) subscribes to a unique, pre-defined alert topic. This ensures that each client only receives messages relevant to their role.

b) Node-RED Flow and Business Logic

The core intelligence of the system resides in the Node-RED flow.

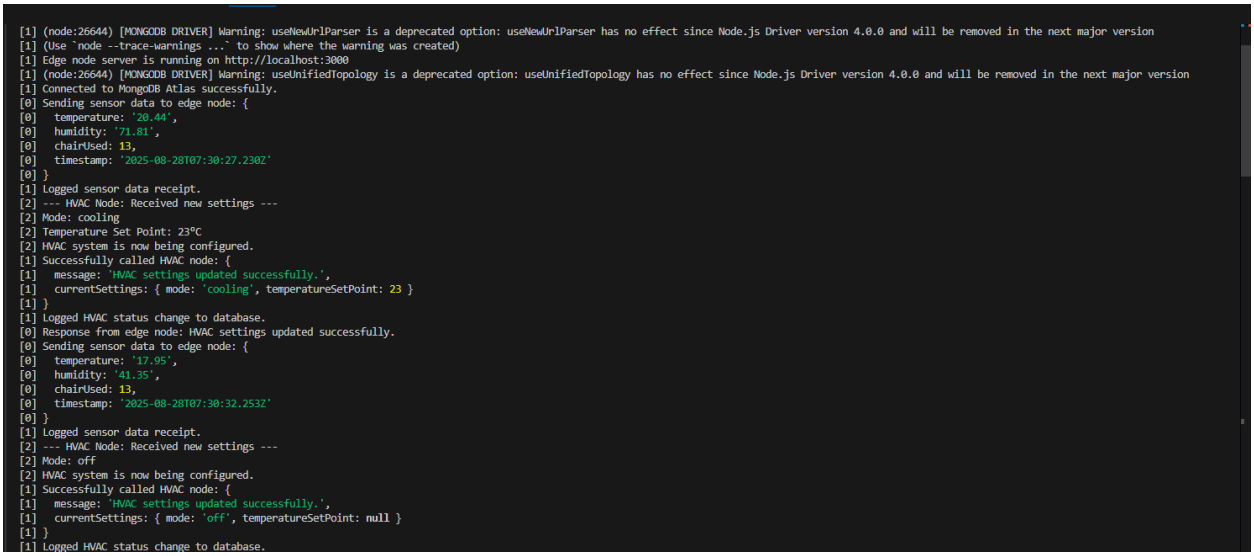
- **MQTT In Node**: This node is configured to subscribe to the broad topic /forest/sensors/#, allowing it to receive data from all sensor types (smoke, heat, wind, humidity) with a single node.
- **Function Node**: This is the brain of the flow. It processes the incoming messages and applies our custom logic.
    - **Flow Context**: To implement the rule that an alert requires multiple factors, we utilized Node-RED's flow context. The flow.get('alertStates') variable acts as a temporary memory, storing the current alert status and sensor level for each sensor type.
    - **Complex Logic**: The code checks the sensor type (msg.topic.split('/')) and updates the alertStates dictionary.
    - **Conditional Alerts**: The script checks two primary conditions:
        1. **General Alert**: If the total number of alerting sensors (totalAlerts >= 2), a general alert is pushed to topics for homeowners, news, and social media.
        2. **Fire Department Alert**: If specific, critical levels are detected (e.g., smokeLevel >= 50 && heatLevel >= 80), a more detailed alert is pushed to the fire department's topic.
    - **Returning Multiple Messages**: The function returns an array of messages (return [alertsToSend];), which Node-RED automatically handles by sending each message to its respective topic.
- **MQTT Out Nodes**: Two separate mqtt out nodes are used. They receive the messages from the function node and publish them to their assigned topics (e.g., /alert/local_homeowners, /alert/fire_departments), completing the data flow.

c) Automation

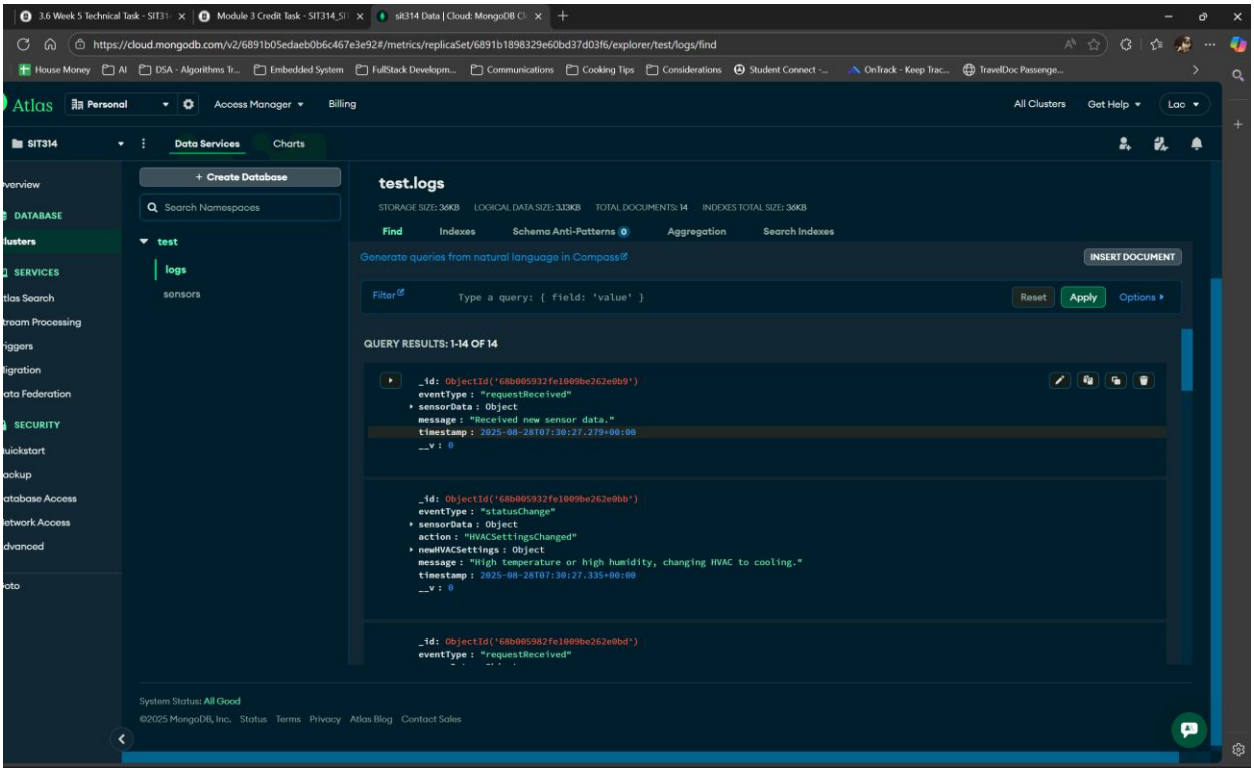
To streamline testing and deployment, we used concurrently within the package.json file. This allows all sensor and client scripts to be started simultaneously with a single command (npm start), eliminating the need to manage multiple terminal windows.

Module 3 – Enterprise Architecture

Screenshots:



Picture 14 – Logs generated from local-side.



Picture 15 – Logs stored on MongoDB Atlas.

Explanation:

1. System Architecture

The application is structured with three main components:

- **Room Sensor Node:** This component simulates several sensors that generate environmental data in JSON format, including room temperature, humidity, and the number of chairs being used.
- **Edge Node:** This acts as the central hub of the system. It receives data from the sensor nodes, analyzes it based on predefined logic, and decides whether to send a command to the HVAC node to change the settings. It is also responsible for logging all requests and status changes to a MongoDB database.
- **HVAC (Heating, Ventilation, and Air Conditioning) Node:** This node is called by the edge node to set the current configuration of the room's HVAC system. It simulates the actions of a physical air-conditioning unit.

2. Implementation and Code

The following sections provide the code for each component of the system.

1/ Room Sensor Node (roomSensorNode.js)

This script uses a function to generate random values for temperature, humidity, and chair usage. It then sends this data to the edge node's API endpoint every 5 seconds.

```
// roomSensorNodes.js

const { generateSensorData } = require('./sensorSimulator');

const axios = require('axios');


const edgeNodeUrl = 'http://localhost:3000/sensor-data';


const sendDataToEdgeNode = async () => {

  const sensorData = generateSensorData();

  console.log('Sending sensor data to edge node:', sensorData);
```

```
  try {
    const response = await axios.post(edgeNodeUrl, sensorData, {
      headers: {
        'Content-Type': 'application/json'
      }
    });
    console.log('Response from edge node:', response.data);
  } catch (error) {
    console.error('Error sending data to edge node:', error.message);
  }
};

// Simulate sending data every 5 seconds
setInterval(sendDataToEdgeNode, 5000);
```

2 - Edge Node (edgeNode.js)

This is the core of the application. It uses Express to create a server and Mongoose to connect to the MongoDB Atlas database. It listens to sensor data, applies logic to decide on HVAC settings, and logs all events. Logging is a critical part of this task. The code has been optimized to only call the HVAC node if the settings need to be changed, thus reducing unnecessary API calls.

```
// edgeNode.js
const express = require('express');
const mongoose = require('mongoose');
```

```
const Log = require('./models/Log');
const axios = require('axios');

const app = express();
const port = 3000;
let currentHVACMode = 'off';
let currentTemperatureSetPoint = null;
app.use(express.json());

const dbURI = 'YOUR_MONGODB_ATLAS_CONNECTION_STRING';
mongoose.connect(dbURI, { useNewUrlParser: true, useUnifiedTopology: true })
.then(() => console.log('Connected to MongoDB Atlas successfully.'))
.catch(err => console.error('Connection error to MongoDB Atlas:', err));

app.post('/sensor-data', async (req, res) => {
  const { temperature, humidity, chairsUsed } = req.body;
  try {
    const logReceived = new Log({
      eventType: 'requestReceived',
      sensorData: { temperature, humidity, chairsUsed },
      message: 'Received new sensor data.'
    });
    await logReceived.save();
    console.log('Logged sensor data receipt.');
  } catch (err) {
    console.error('Error logging sensor data:', err);
```

```
  }


  let newHVACMode;
  let newTemperatureSetPoint;
  let logMessage;
  if (temperature > 26 || humidity > 50) {
    newHVACMode = 'cooling';
    newTemperatureSetPoint = 23;
    logMessage = 'High temperature or high humidity, changing HVAC to cooling.';
  } else {
    newHVACMode = 'off';
    newTemperatureSetPoint = null;
    logMessage = 'Low temperature or normal humidity, turning off HVAC.';
  }


  if (newHVACMode === currentHVACMode && newTemperatureSetPoint ===
currentTemperatureSetPoint) {
    console.log('HVAC mode unchanged. No action needed.');
    const logNoChange = new Log({
      eventType: 'statusUnchanged',
      sensorData: { temperature, humidity, chairsUsed },
      message: 'HVAC settings remain the same as no threshold was crossed.'
    });
    await logNoChange.save();
    return res.status(200).send('HVAC settings are already in the correct state.');
  }
```

```
  currentHVACMode = newHVACMode;
  currentTemperatureSetPoint = newTemperatureSetPoint;

  const hvacUrl = 'http://localhost:3001/set-hvac';
  const newHVACSettings = { mode: newHVACMode, temperatureSetPoint:
newTemperatureSetPoint };

  try {
    const hvacResponse = await axios.post(hvacUrl, newHVACSettings);
    console.log('Successfully called HVAC node:', hvacResponse.data);

    const logStatusChange = new Log({
      eventType: 'statusChange',
      sensorData: { temperature, humidity, chairsUsed },
      action: 'HVACSettingsChanged',
      newHVACSettings: newHVACSettings,
      message: logMessage
    });
    await logStatusChange.save();
    console.log('Logged HVAC status change to database.');

    res.status(200).send('HVAC settings updated successfully.');
  } catch (error) {
    console.error('Failed to call HVAC node:', error.message);
    res.status(500).send('Failed to update HVAC settings.');
```

```
  }
});
app.listen(port, () => console.log(` Edge node server is running on http://localhost:${port}`));
```

3 - HVAC Node (hvacNode.js)

This script creates a server to receive command from the edge node. It simply logs the received settings to the console to simulate the HVAC unit being configured.

```
// hvacNode.js
const express = require('express');
const app = express();
const port = 3001;

app.use(express.json());

app.post('/set-hvac', (req, res) => {
  const { mode, temperatureSetPoint } = req.body;
  console.log('--- HVAC Node: Received new settings ---');
  console.log(` Mode: ${mode}`);
  if (temperatureSetPoint) {
    console.log(` Temperature Set Point: ${temperatureSetPoint}°C`);
  }
  console.log('HVAC system is now being configured.');

  res.status(200).send({
```

```
    message: 'HVAC settings updated successfully.',
    currentSettings: { mode, temperatureSetPoint }
  });
});
app.listen(port, () => console.log(` HVAC node server is running on
http://localhost:${port}` ));
```

Module 4 – Event-Driven Architectures

A – Task Summary

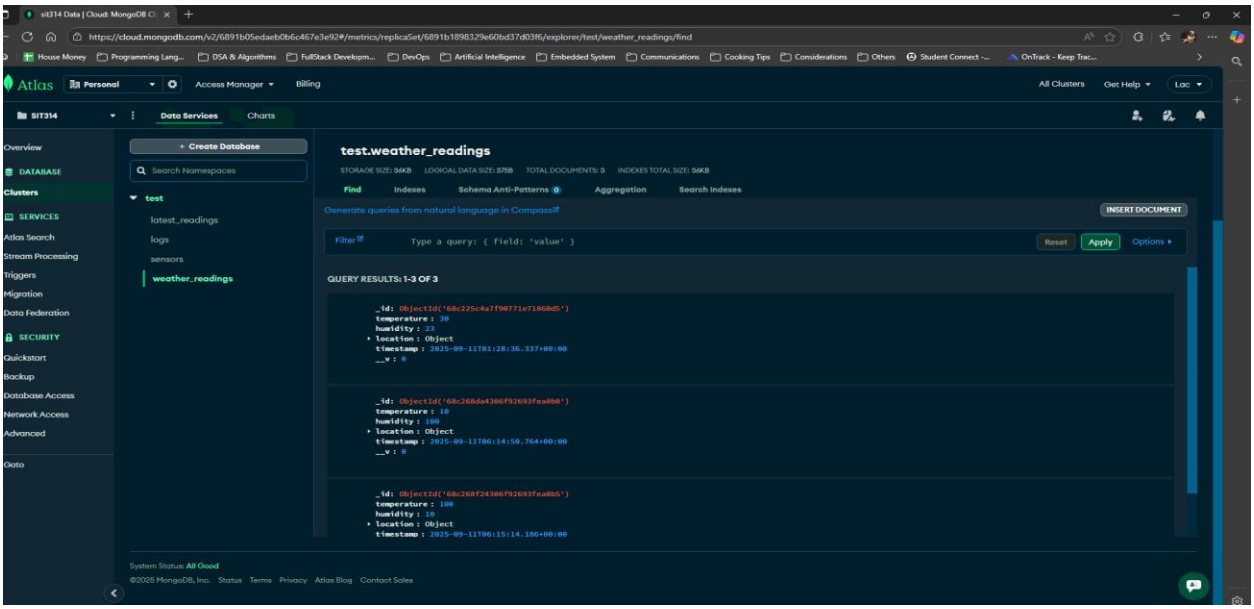
- This task involved building a comprehensive local weather web service that receives, stores, and provides both local sensor data and public weather information.
- The solution successfully integrates a RESTful API with a MongoDB database and an external weather service to fulfill all the task requirements.
- The backend is designed for scalability and efficiency, while a simple frontend client demonstrates all functionalities.

B - Key Components and Functionality

1. Database and Schemas

To manage the data effectively, two separate Mongoose schemas were developed, each corresponding to a dedicated MongoDB collection:

- **“weather_readings” Collection:**
    - This collection serves as a historical log for all local sensor data.
    - The schema, defined in WeatherReading.js, captures the temperature, humidity, timestamp, and location of each reading.
    - This approach allows for long-term data storage and analysis.



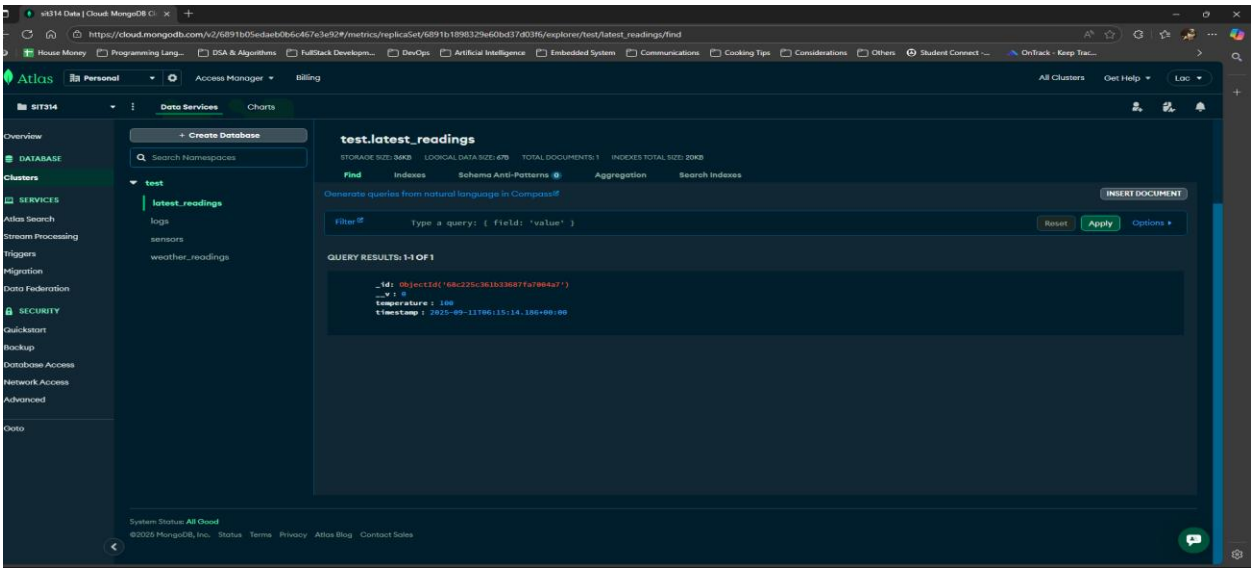
Picture 1 – “weather_readings” collection.

```
const mongoose = require('mongoose');

const weatherReadingSchema = new mongoose.Schema({

    temperature: {
        type: Number,
        required: true
    },

    humidity: {
        type: Number,
        required: true
    },

    timestamp: {
        type: Date,
        default: Date.now,
        required: true
    },

    location: {
        latitude: {
            type: Number,
            required: true,
        },
        // Corrected spelling from 'longtitude' to 'longitude'
        longitude: {
```

```
            type: Number,
            required: true,
        }
    }

});

module.exports = mongoose.model('weather_reading', weatherReadingSchema);
```

Code 1 – Schema code for “weather_readings” collection on MongoDB.

- **“latest_reading” Collection:**
    - This specialized collection stores only a single document containing the most recent temperature reading.
    - This design choice is a key optimization that allows for extremely fast retrieval of the latest data without needing to query the entire historical log.



Picture 2 – “latest_readings” collection

```
// latestReading.js
const mongoose = require('mongoose');

// This schema is specifically for storing the single latest temperature reading
const latestReadingSchema = new mongoose.Schema({
  temperature: {
    type: Number,
    required: true
  },
  timestamp: {
    type: Date,
    default: Date.now,
    required: true
  }
});

module.exports = mongoose.model('latest_reading', latestReadingSchema);
```

Code 2 – Schema code for "lastest_readings" collection on MongoDB

2. API Endpoints (CRUD Operations)

The core of the web service is the weatherService_api.js file, which provides a full suite of API endpoints to manage the data and interact with external services. The endpoints are as follows:

- **POST /api/weather: (CREATE)** Accepts new sensor readings (temperature, humidity, and location) from a client. It saves a new document to the weather_reading collection and simultaneously updates the single document in the latest_reading collection.
- **GET /api/weather: (READ)** Retrieves and returns all historical sensor readings from the weather_reading collection.
- **PUT /api/weather/:id: (UPDATE)** Allows for the modification of an existing sensor reading by its unique ID.
- **DELETE /api/weather/:id: (DELETE)** Permanently removes a specific sensor reading by its unique ID.
- **GET /api/weather/latest: (READ)** Provides an optimized endpoint to quickly retrieve the most recent temperature reading from the latest_reading collection.

- **GET /api/weather/location/:place: (READ)** Integrates the weather-js module to fetch current weather data for a specified location from an external service. This data is not stored in the database.

```
// api.js
// This file sets up the backend API for the weather service.
// It uses Express for routing, Mongoose for MongoDB interaction, and weather-js
for external API calls.

const express = require('express');
const mongoose = require('mongoose');
const bodyParser = require('body-parser');
const cors = require('cors');
const weather = require('weather-js');

const app = express();
const PORT = 3000;

// --- Middleware ---

app.use(cors());
app.use(bodyParser.json());

// --- MongoDB Connection ---
// NOTE: Using a hardcoded URI for demonstration purposes. In a real application,
use environment variables.
const mongoUri =
'mongodb+srv://tamlac20121996:PuIDWhd25HIqpj07@sit314.tchyumf.mongodb.net';
mongoose.connect(mongoUri, { useNewUrlParser: true, useUnifiedTopology: true })
    .then(() => console.log('Connected to MongoDB'))
    .catch(err => console.error('Could not connect to MongoDB:', err));

// --- Import Mongoose Models ---
// Corrected import to match the 'LastestReading.js' filename.
const WeatherReading = require('./models/WeatherReading.js');
const LatestReading = require('./models/LastestReading.js');

// --- API Endpoints (CRUD Operations) ---

// 1. CREATE: Submit a new sensor reading
app.post('/api/weather', async (req, res) => {
    try {
        // Corrected 'longtitude' field to 'longitude' to match the client input
and the updated schema.
        const { temperature, humidity, latitude, longitude } = req.body;
```

```
        if (temperature === undefined || humidity === undefined || latitude === undefined || longitude === undefined) {
            return res.status(400).json({ message: 'Missing required fields: temperature, humidity, latitude, longitude' });
        }

        const newReading = new WeatherReading({ temperature, humidity, location: { latitude, longitude } });
        await newReading.save();

        // Update the single latest reading document
        // Use findOneAndUpdate with upsert: true to create the document if it doesn't exist.
        await LatestReading.findOneAndUpdate({}, { temperature, timestamp: newReading.timestamp }, {
            upsert: true,
            new: true
        });

        res.status(201).json({ message: 'Sensor reading submitted successfully', data: newReading });
    } catch (error) {
        // Log the full error to help with debugging
        console.error('Error submitting reading:', error);
        res.status(500).json({ message: 'Error submitting reading', error: error.message });
    }
});

// 2. READ: Get all weather readings
app.get('/api/weather', async (req, res) => {
    try {
        const readings = await WeatherReading.find().sort({ timestamp: -1 });
        res.status(200).json(readings);
    } catch (error) {
        console.error('Error retrieving readings:', error);
        res.status(500).json({ message: 'Error retrieving readings', error: error.message });
    }
});

// 3. READ: Get the latest temperature reading
app.get('/api/weather/latest', async (req, res) => {
    try {
        const latest = await LatestReading.findOne();
```

```
        if (!latest) {
            return res.status(404).json({ message: 'No latest reading found' });
        }
        res.status(200).json(latest);
    } catch (error) {
        console.error('Error retrieving latest reading:', error);
        res.status(500).json({ message: 'Error retrieving latest reading', error:
error.message });
    }
});

// 4. READ: Get weather data for a specific map location using weather-js
app.get('/api/weather/location/:place', (req, res) => {
    const { place } = req.params;
    weather.find({ search: place, degreeType: 'C' }, (err, result) => {
        if (err) {
            console.error(err);
            return res.status(500).json({ message: 'Error retrieving weather data
for location' });
        }
        if (result.length === 0) {
            return res.status(404).json({ message: 'Location not found' });
        }
        res.status(200).json(result[0]);
    });
});

// 5. UPDATE: Update a specific reading by ID
app.put('/api/weather/:id', async (req, res) => {
    try {
        const { id } = req.params;
        const updateData = req.body;
        const updatedReading = await WeatherReading.findByIdAndUpdate(id,
updateData, { new: true });
        if (!updatedReading) {
            return res.status(404).json({ message: 'Reading not found' });
        }

        // Check if the updated reading is the latest one
        const latest = await LatestReading.findOne();
        if (latest && latest.timestamp <= updatedReading.timestamp) {
            latest.temperature = updatedReading.temperature;
            latest.timestamp = updatedReading.timestamp;
            await latest.save();
        }
```

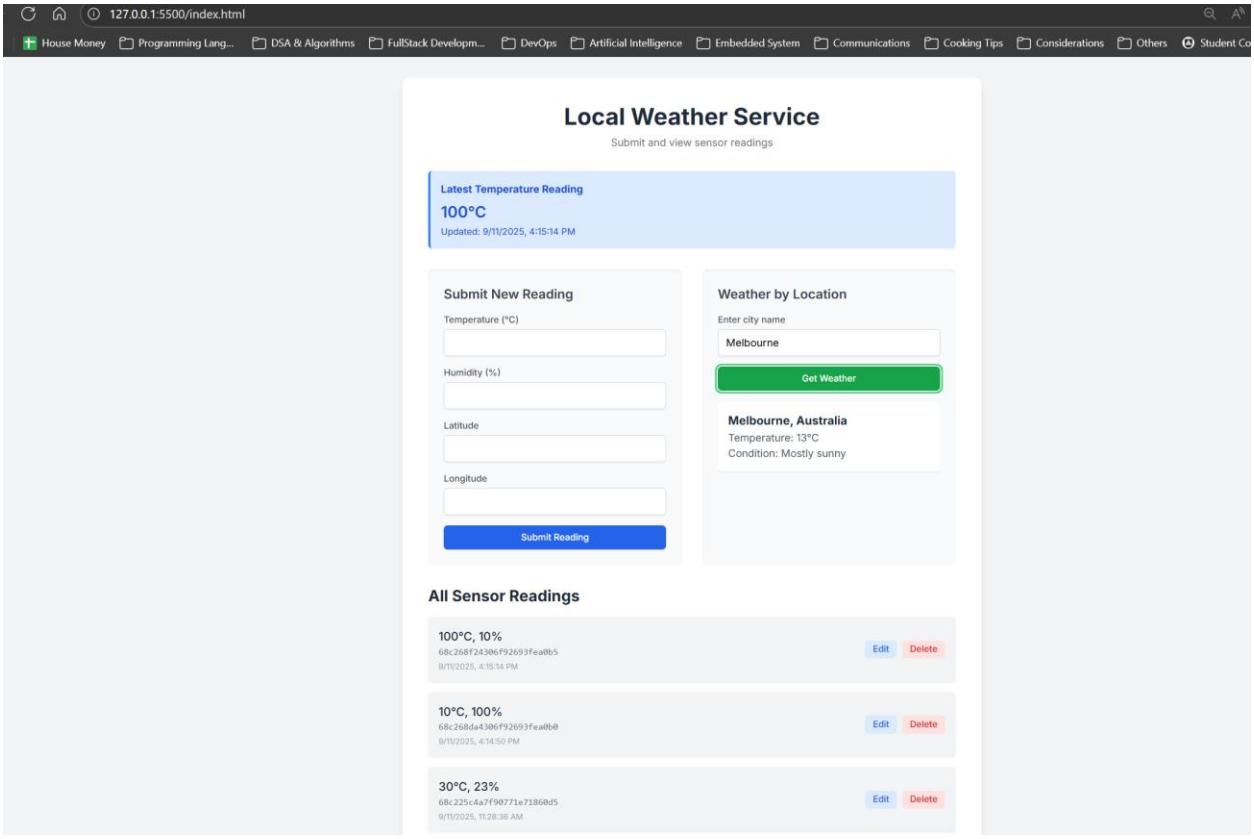
```
        res.status(200).json({ message: 'Reading updated successfully', data:
updatedReading });
    } catch (error) {
        console.error('Error updating reading:', error);
        res.status(500).json({ message: 'Error updating reading', error:
error.message });
    }
});

// 6. DELETE: Delete a specific reading by ID
app.delete('/api/weather/:id', async (req, res) => {
    try {
        const { id } = req.params;
        const deletedReading = await WeatherReading.findByIdAndDelete(id);
        if (!deletedReading) {
            return res.status(404).json({ message: 'Reading not found' });
        }

        // You might need to re-evaluate the latest reading if the deleted one
was the latest
        // For simplicity, this example doesn't handle that edge case.
        res.status(200).json({ message: 'Reading deleted successfully', data:
deletedReading });
    } catch (error) {
        console.error('Error deleting reading:', error);
        res.status(500).json({ message: 'Error deleting reading', error:
error.message });
    }
});

// Start the server
app.listen(PORT, () => {
    console.log(`Server is running on http://localhost:${PORT}`);
});
```

Code 3 – “weatherService_api.js”

3. Client

A simple web client (index.html) was developed to demonstrate the functionality of all the API endpoints. The client features forms to submit new readings, buttons to update or delete existing ones, and an interface to query for weather at a specific location using the integrated weather-js functionality.

This project successfully demonstrates the ability to build a robust, scalable, and modular web service that fulfills a variety of data management and integration requirements.



Picture 3 – Client UI for direct interaction.

Module 5 – Scalable Deployment

A - Report on Load-Balanced API Deployment

This report details the deployment of a load-balanced MongoDB API on Amazon EC2, fulfilling the requirements for the Module 5 credit task.

The system aims to collect sensor data from clients, route it through an Application Load Balancer (ALB), and store it in a MongoDB Atlas database. The core API is replicated across two separate EC2 instances to enable load balancing.

The deployment encountered two primary errors: a Request error: Error and an HTTP 404 Cannot POST / error. The Request error indicated a network issue, preventing the client

from even reaching the load balancer. This was due to misconfigured Inbound rules in the security groups.

To fix this, I created separate security groups for the Load Balancer and the EC2 instances, ensuring the ALB was accessible from the internet on port 3000, while the EC2 instances were only accessible from the ALB's security group, a best practice for secure and reliable communication.

The 404 Cannot POST / error, which persisted even after network connectivity was resolved, pointed to a server-side routing problem. Through systematic debugging, the root causes were identified:

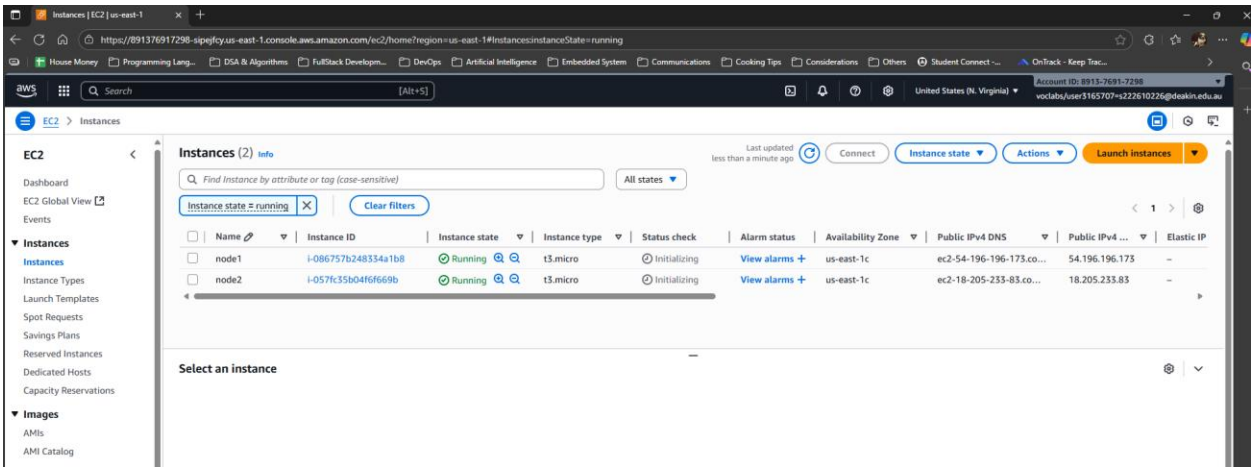
- **Process Conflicts:** It was discovered that multiple Node.js processes were running simultaneously on the EC2 instance, causing port conflicts which likely came from previous technical tasks.
- This was resolved by stopping all conflicting processes with the kill command.

After resolving these issues, the system successfully processed POST requests and stored the data in the designated sit314 database on MongoDB Atlas.

Video Link: https://deakin.au.panopto.com/Panopto/Pages/Viewer.aspx?id=46c9ca23-00e0-461e-b937-b35b0055c022

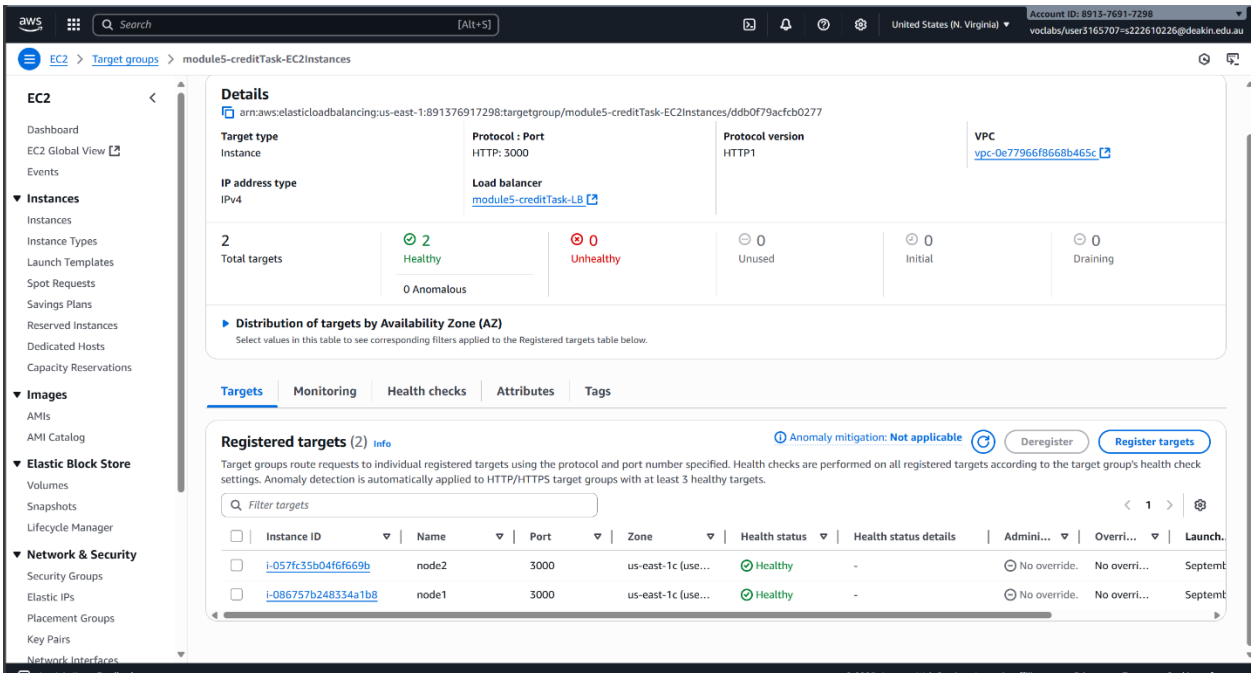
B – Steps Taken & Evidence

1. Launched 2 EC2 instances based on the Pass-task AMI and named them as “node1” & “node2” (Picture 1)
    - However, the inbound rules of security group for both instances were set to:
        i. Custom TCP – Source = “security group ID of Load Balancer” (did this step after security group for ALB was created)
        ii. SSH – Source = “myIP” / or everywhere

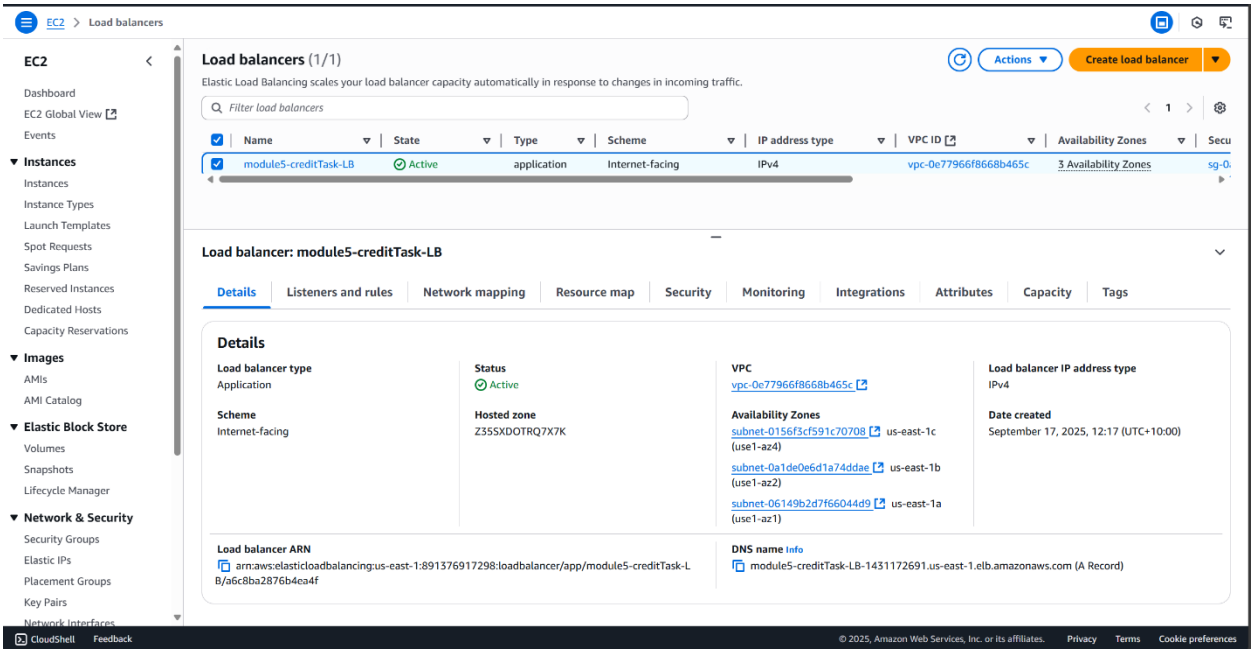


Picture 1: Generated 2 EC2 instances, from Pass-task AMI, “node1” & “node2”

2. Created a Load-Balancer as the following stats:
    - Load balancer types: “Application Load Balancer”
    - Load balancer name: “module5-creditTask-LB”
    - Availability Zones and subnets: Pick any 3 from the list
    - Security groups: Created a new security group with the following stats:
        i. Name: SG-ALB
        ii. Inbound Rules:
            - HTTP – Source “Anywhere IPv4”
            - HTTPS – Source “Anywhere IPv4”
    - Listeners and routing:
        i. Protocol: “HTTP” – Port: “3000”
        ii. Routing action: “Forward to target groups”
            - Created a target group that consisted of “node1” & “node2” with similar protocol and port number (Picture 2)



Picture 2 – Created a target group that consisted of “node1” & “node2”



Picture 3 – Created an ALB called “module5-creditTask-LB”

3. Connected to these 2 EC2 instances through Windows 11 Command Prompt with provided ssh from each EC2.

4. For both EC2 instances:
    - Removed the crontab job from previous tasks
    - Mkdir “module5_credit”
    - Npm init -y
    - Npm install express mongoose
    - Nano server.js and added the following code:

```
// server.js
const express = require('express');
const mongoose = require('mongoose');

const Sensor = require('./sensor');

const app = express();
const port = 3000;

// Middleware to parse incoming JSON data
app.use(express.json());

// MongoDB Atlas connection string
const serveraddress =
'mongodb+srv://tamlac20121996:PuIDWhd25HIqpj07@sit314.tchyumf.mongodb.net/sit314'
;

// Connect to MongoDB
mongoose.connect(serveraddress)
  .then(() => console.log('Connected to MongoDB Atlas'))
  .catch(err => console.error('Could not connect to MongoDB Atlas...', err));

// Health check endpoint for the Load Balancer
app.get('/health', (req, res) => {
  res.status(200).send('API is healthy');
});

// GET all sensor readings
app.get('/', async (req, res) => {
  try {
    const all = await Sensor.find({});
    res.send(all);
  } catch (err) {
    res.status(500).send({ message: 'Error retrieving sensor data', error: err
});
  }
});
```

```
// GET sensor reading by ID
app.get('/:id', async (req, res) => {
  try {
    const sensor = await Sensor.findById(req.params.id);
    if (!sensor) return res.status(404).send('Sensor with the given ID was not
found.');
    res.send(sensor);
  } catch (err) {
    res.status(500).send({ message: 'Error retrieving sensor data by ID', error:
err });
  }
});

// POST a new sensor reading
app.post('/', async (req, res) => {
  try {
    // Read the data from the request body
    const newSensor = new Sensor({
      name: req.body.name,
      address: req.body.address,
      time: req.body.time,
      temperature: req.body.temperature
    });

    const result = await newSensor.save();
    console.log("Saving Sensor reading to Database");
    console.log(result);
    res.status(201).send(result);
  } catch (err) {
    res.status(400).send({ message: 'Invalid data provided', error: err });
  }
});

// PUT to update an existing sensor reading
app.put('/:id', async (req, res) => {
    try {
        const updatedSensor = await Sensor.findByIdAndUpdate(req.params.id, {
            name: req.body.name,
            address: req.body.address,
            time: req.body.time,
            temperature: req.body.temperature
        }, { new: true });
```

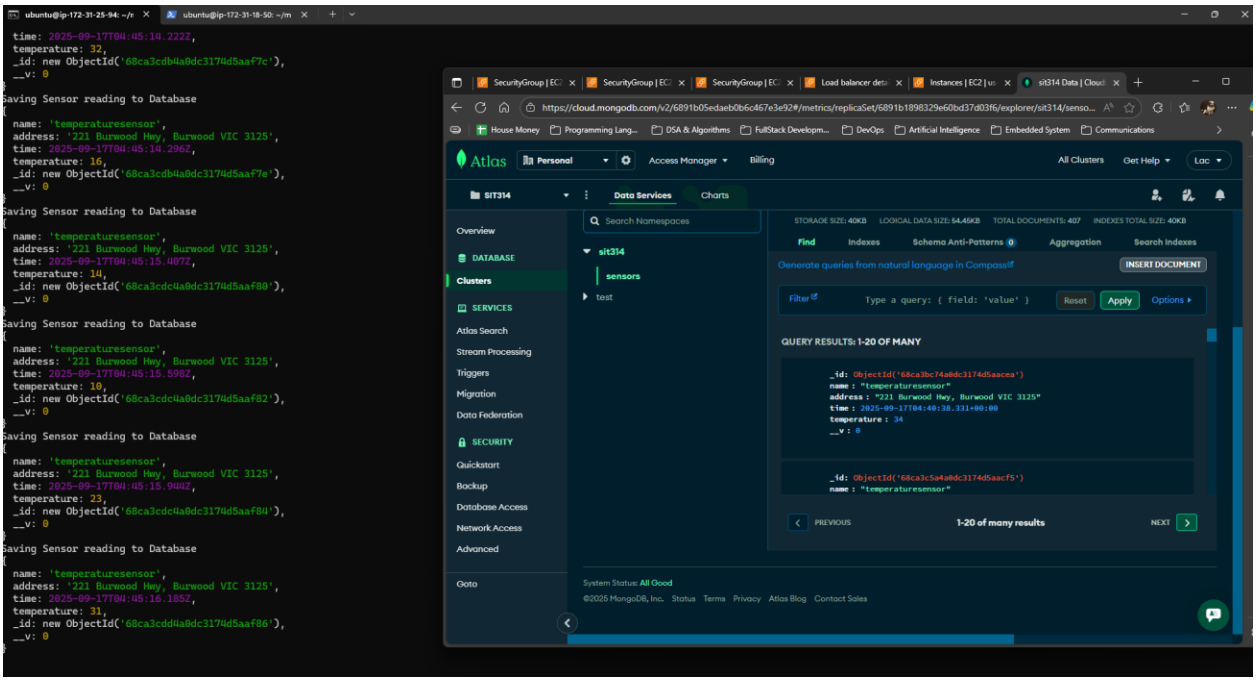
```
        if (!updatedSensor) return res.status(404).send('Sensor with the given ID
was not found.');
        res.send(updatedSensor);
    } catch (err) {
        res.status(400).send({ message: 'Error updating sensor data', error: err
});
    }
});

// DELETE a sensor reading
app.delete('/:id', async (req, res) => {
    try {
        const sensor = await Sensor.findByIdAndDelete(req.params.id);
        if (!sensor) return res.status(404).send('Sensor with the given ID was
not found.');
        res.status(200).send(sensor);
    } catch (err) {
        res.status(500).send({ message: 'Error deleting sensor data', error: err
});
    }
});


app.listen(port, () => {
  console.log(`API listening on port ${port}`);
});
```

- Nano sensor.js and added the following code for sensor Mongodb-schema:

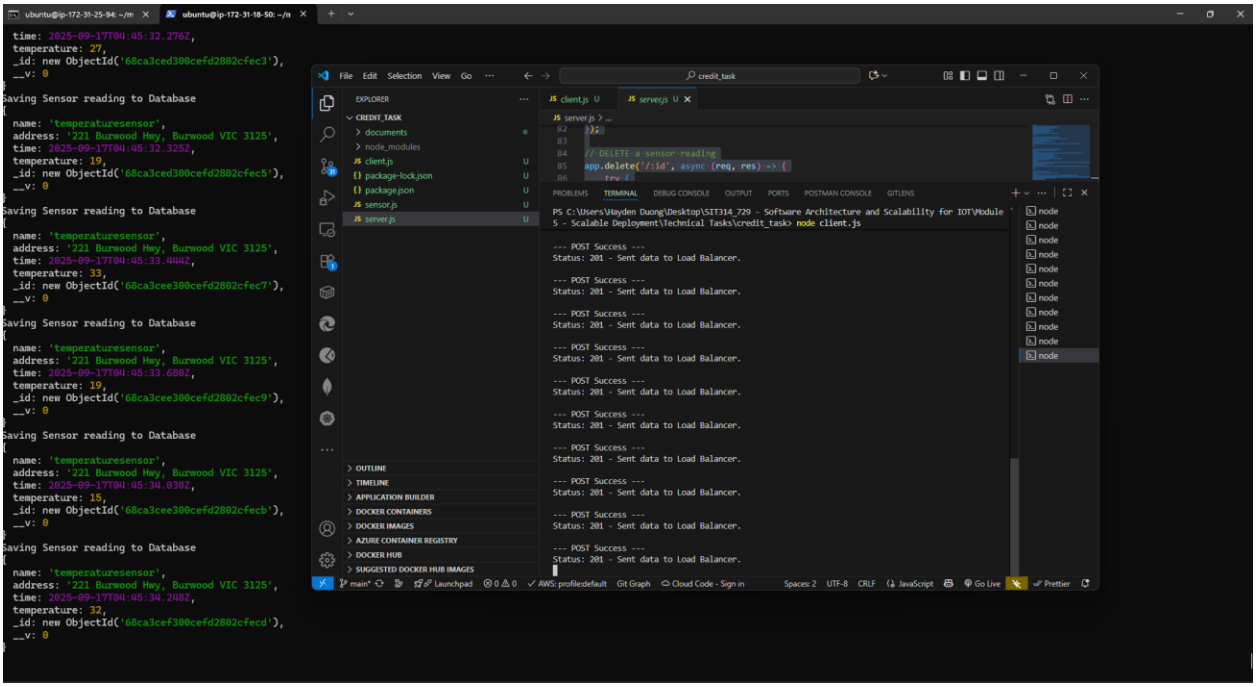
```
const mongoose = require('mongoose');

module.exports = mongoose.model('Sensor', new mongoose.Schema({
  id: Number,
  name: String,
  address: String,
  time: Date,
  temperature: Number
}));
```

5. On local computer:
    - Mkdir “credit_task”
    - Npm init -y
    - Npm install axios
    - Create “client.js” and added the following codes:

```
// client.js
const axios = require('axios');

// The URL consists of the load balancer DNS name and the port number
const baseURL = `http://module5-creditTask-LB-1431172691.us-east-
1.elb.amazonaws.com:3000`;

// Function to send POST request with sensor data
// The sensor data includes name, address, current time, and a random temperature
between 10 and 40
const postSensorData = async () => {
  const newSensorData = {
    name: "temperaturesensor",
    address: "221 Burwood Hwy, Burwood VIC 3125",
    time: new Date(),
    temperature: Math.floor(Math.random() * (40 - 10) + 10)
  };

  try {
    const response = await axios.post(baseURL, newSensorData);
    console.log(`\n--- POST Success ---`);
    console.log(`Status: ${response.status} - Sent data to Load Balancer.`);
  } catch (error) {
    console.log(`\n--- POST Error ---`);
    if (error.response) {
      console.log(`Status: ${error.response.status}`);
      console.log(`Data: ${error.response.data}`);
    } else {
      console.log(`Request error: ${error.message}`);
    }
  }
};

// Set an interval to send POST requests every 2 seconds
const interval = 2000; // 2000 ms = 2 seconds
console.log(`Starting to send POST requests to the Load Balancer every ${interval
/ 1000} seconds...`);
setInterval(postSensorData, interval);
```



Picture 4 – EC2 “node1” received and sent the data to MongoDB Atlas.



Picture 5 – EC2 “node2” received and sent the data to MongoDB Atlas after 7-8 “client.js” were running.

Module 6 – Scalable IoT Security

Case Study 1: The Mirai Botnet

A - Overview of the Incident

In 2016, the Mirai botnet emerged as one of the most disruptive demonstrations of IoT insecurity. Mirai infected thousands of consumer-grade internet-connected devices—such as digital video recorders (DVRs), IP cameras, and routers—transforming them into a massive botnet used to execute distributed denial-of-service (DDoS) attacks.

The most notable attack targeted Dyn, a major domain name system (DNS) provider. This caused internet outages across North America and parts of Europe, rendering popular services like Twitter, Netflix, and Reddit inaccessible (Antonakakis et al., 2017; Krebs, 2016).

B - Technical Issue

The root vulnerability exploited by Mirai was the widespread use of weak or default login credentials. The malware scanned the internet for devices with open Telnet ports and attempted to access them using a list of more than 60 hardcoded username–password combinations.

Once compromised, the devices became remotely controlled nodes in the botnet. The absence of secure firmware update mechanisms further exacerbated the issue, as many devices could not be patched once infected (Antonakakis et al., 2017).

C - Loss or Harm Caused

The attack disrupted internet services for millions of users and caused financial and reputational harm to numerous organizations.

For example, Dyn experienced significant operational strain, while businesses relying on its DNS services faced customer dissatisfaction and lost revenue.

The attack underscored how IoT vulnerabilities could undermine the stability of global internet infrastructure (Symantec, 2017).

D - Lessons Learned

Manufacturers should have enforced mandatory password changes during device setup, avoided hardcoded credentials, and provided secure, automated firmware updates. Internet service providers could have implemented better anomaly detection to mitigate large-scale DDoS activity.

End-users also needed greater awareness of securing their devices through unique passwords and regular updates (US-CERT, 2016). The Mirai botnet remains a pivotal case study in the necessity of “security by default” within IoT ecosystems.

Case Study 2: BrickerBot Malware

A - Overview of the Incident

In 2017, security researchers discovered a unique strain of malware called BrickerBot. Unlike Mirai, which recruited devices for botnets, BrickerBot performed Permanent Denial-of-Service (PDoS) attacks, effectively “bricking” IoT devices by corrupting their storage and rendering them irreparable.

The anonymous creator, self-identified as “Janit0r,” claimed to be acting as a vigilante aiming to reduce the pool of insecure devices exploitable by malicious actors (Cimpanu, 2017; CISA, 2017).

B - Technical Issue

BrickerBot exploited the same weaknesses as Mirai: open Telnet ports and default or hardcoded credentials. Once it gained access, the malware issued destructive commands, including wiping file systems, corrupting storage, and disabling network connectivity.

Without recovery mechanisms, most infected devices became permanently unusable. This highlighted the vulnerability of IoT hardware lacking resilience against malicious reconfiguration (Radware, 2017).

C - Loss or Harm Caused

The attack reportedly destroyed millions of IoT devices worldwide, including routers, security cameras, and industrial equipment (Cimpanu, 2017).

Unlike DDoS attacks, which cause temporary service disruptions, BrickerBot inflicted permanent hardware loss, leading to costly device replacements for both consumers and enterprises.

This created a new threat model where IoT insecurity could lead to irreversible physical damage rather than just software compromise (Kaspersky, 2017).

D - Lessons Learned

Manufacturers should have eliminated insecure default settings, disabled Telnet by default, and implemented secure firmware with mechanisms for recovery.

The incident highlights the importance of designing IoT systems with resilience, redundancy, and fail-safe mechanisms.

Users also bear responsibility by practicing strong credential management and restricting unnecessary remote access (F5 Labs, 2017).

BrickerBot demonstrated how malicious intent—even with claimed "good intentions"—can cause widespread collateral damage if IoT devices lack fundamental protections.

Case Study 3: The St. Jude Medical Pacemaker Vulnerability

A - Overview of the Incident

In 2017, the U.S. Food and Drug Administration (FDA) issued a recall for more than 465,000 pacemakers manufactured by Abbott (formerly St. Jude Medical) due to severe cybersecurity vulnerabilities.

The flaw could allow an attacker to remotely access and reprogram a patient's pacemaker, creating potentially life-threatening consequences. Although no actual attacks were confirmed, the discovery raised serious concerns about the security of medical IoT devices (FDA, 2017; ASA, 2017).

B - Technical Issue

The vulnerability stemmed from weak wireless communication protocols used by the pacemakers to connect with external transmitters.

These communications lacked strong encryption and authentication, enabling a nearby attacker with commercially available equipment to intercept signals and send malicious commands. Such commands could drain the pacemaker's battery or alter pacing settings, directly endangering patient safety (Maisel & Kohno, 2010).

C - Loss or Harm Caused

Although no patients were reported harmed, the potential consequences included severe health risks such as irregular heart rhythms, device failure, or even death.

The recall required costly firmware updates and eroded trust in medical device manufacturers.

Public awareness of the vulnerabilities in life-sustaining IoT devices grew significantly, raising ethical and regulatory questions about cybersecurity in healthcare (FDA, 2017).

D - Lessons Learned

This incident highlights the necessity of adopting a “security by design” philosophy in medical IoT.

Manufacturers should have integrated robust encryption, mutual authentication, and intrusion detection systems into device communication.

Regulatory bodies, such as the FDA, have since emphasized premarket cybersecurity assessments and continuous post-market monitoring.

Proactive threat modeling and penetration testing could have prevented such vulnerabilities from reaching patients (Halperin et al., 2008).

References

American Society of Anesthesiologists. (2017, August 31). *Implantable cardiac pacemakers by Abbott (formerly St. Jude Medical): Safety communication - Firmware update to address cybersecurity vulnerabilities*. ASA. https://www.asahq.org/advocacy-and-asapac/fda-and-washington-alerts/fda-alerts/2017/08/implantable-cardiac-pacemakers-by-abbott-firmware-update-cybersecurity-vulnerabilities

Antonakakis, M., April, T., Bailey, M., Bernhard, M., Bursztein, E., et al. (2017). *Understanding the Mirai botnet*. In *26th USENIX Security Symposium* (pp. 1093–1110). USENIX Association. https://www.usenix.org/system/files/conference/usenixsecurity17/sec17-antonakakis.pdf

Cimpanu, C. (2017, December 11). BrickerBot author retires claiming to have bricked over 10 million IoT devices. *BleepingComputer*. https://www.bleepingcomputer.com/news/security/brickerbot-author-retires-claiming-to-have-bricked-over-10-million-iot-devices/

Cybersecurity and Infrastructure Security Agency. (2017, April 18). *BrickerBot permanent denial-of-service attack (Update A)*. CISA. https://www.cisa.gov/news-events/ics-alerts/ics-alert-17-102-01a

F5 Labs. (2017, December 28). *BrickerBot: Do “good intentions” justify the means—or deliver meaningful results?* F5 Labs. https://www.f5.com/labs/articles/threat-

intelligence/brickerbot-do-good-intentions-justify-the-meansor-deliver-meaningful-results

Halperin, D., Heydt-Benjamin, T. S., Fu, K., Kohno, T., & Maisel, W. H. (2008). Security and privacy for implantable medical devices. *IEEE Pervasive Computing, 7*(1), 30–39. https://doi.org/10.1109/MPRV.2008.16

Kaspersky. (2017, April 25). *BrickerBot: The malware that bricks IoT devices*. Kaspersky Daily. https://www.kaspersky.com/blog/brickerbot-malware/

Krebs, B. (2016, October 21). KrebsOnSecurity hit with record DDoS. *KrebsOnSecurity*. https://krebsonsecurity.com/2016/09/krebsonsecurity-hit-with-record-ddos/

Maisel, W. H., & Kohno, T. (2010). Improving the security and privacy of implantable medical devices. *New England Journal of Medicine, 362*(13), 1164–1166. https://doi.org/10.1056/NEJMp0911293

Radware. (2017, April 25). *BrickerBot: Back with a vengeance*. Radware. https://www.radware.com/security/ddos-threats-attacks/brickerbot-pdos-back-with-vengeance/

Symantec. (2017, January). *The Mirai botnet: Threats and lessons learned*. Symantec Security Response. https://symantec-enterprise-blogs.security.com/

U.S. Food and Drug Administration. (2017, January 9). *FDA confirms cybersecurity vulnerabilities of St. Jude's implantable cardiac devices, Merlin transmitter*. Diagnostic and Interventional Cardiology. https://www.dicardiology.com/article/fda-confirms-cybersecurity-vulnerabilities-st-judes-implantable-cardiac-devices-merlin

US-CERT. (2016, October 14). *Alert (TA16-288A): Heightened DDoS threats posed by Mirai and other botnets*. https://www.cisa.gov/news-events/alerts/2016/10/14/heightened-ddos-threats-posed-mirai-and-other-botnets